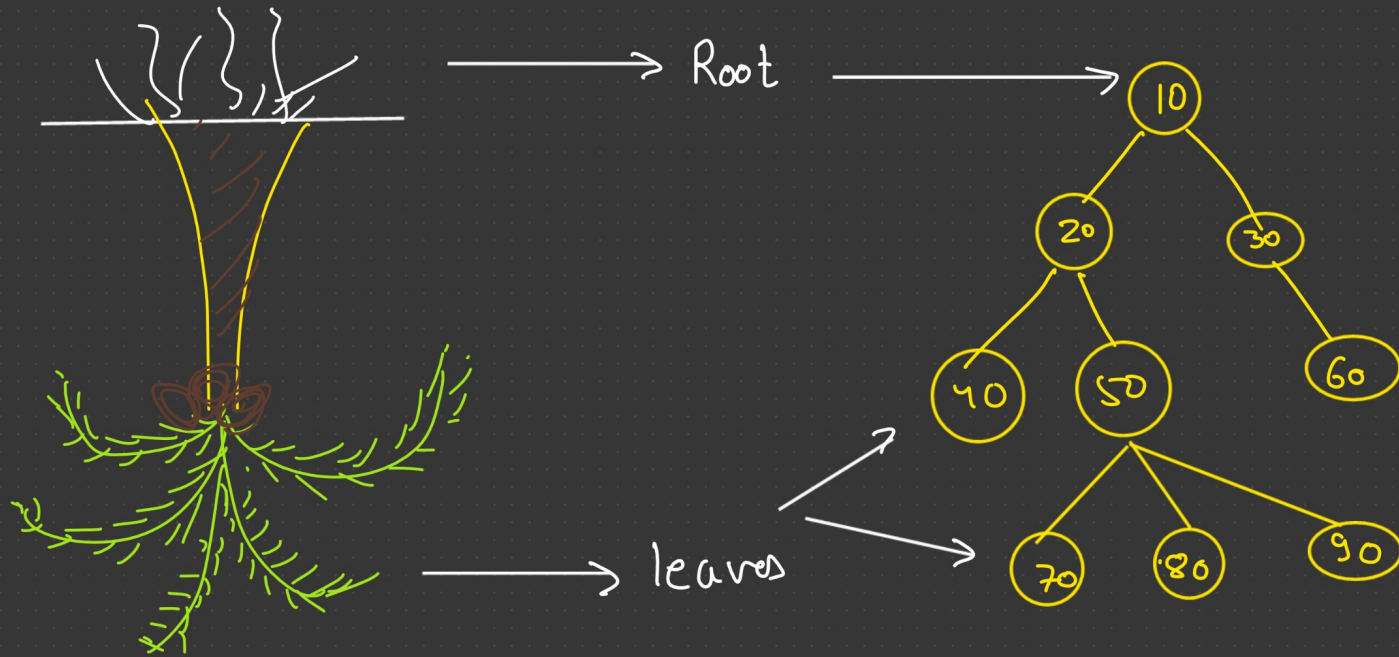
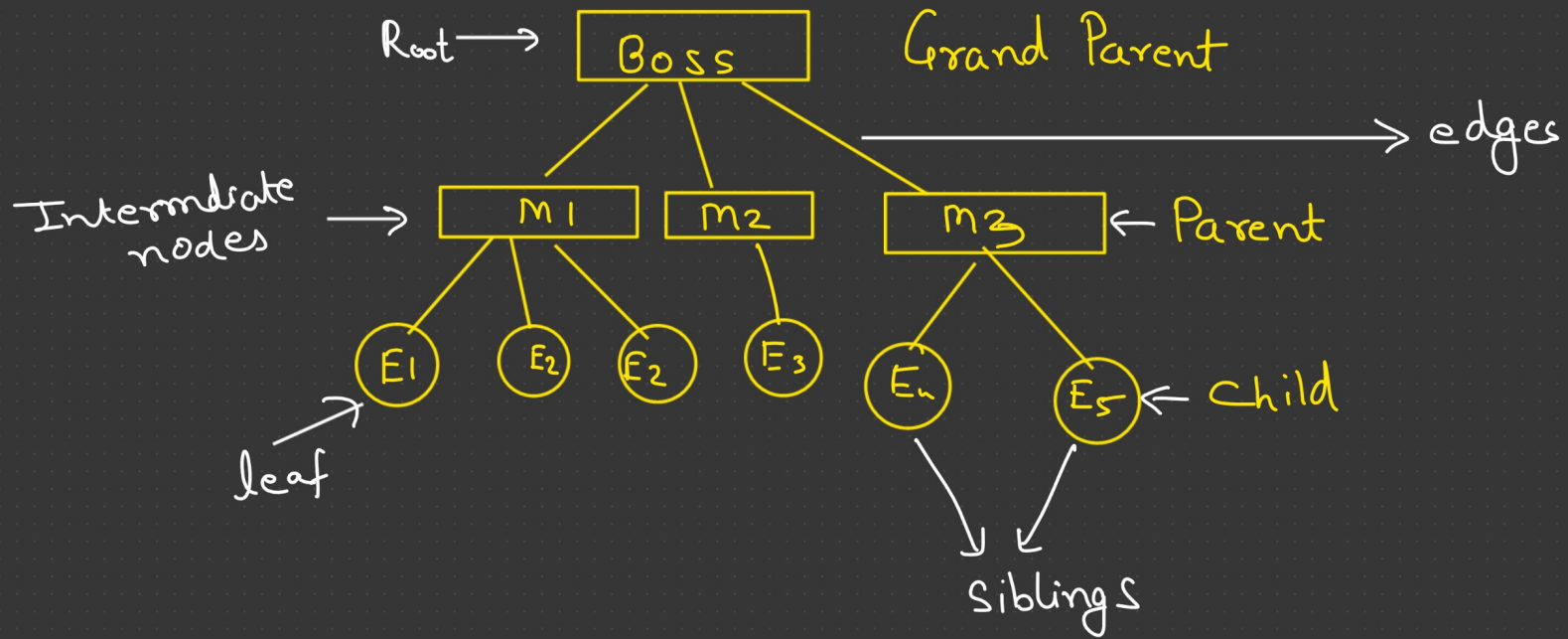


Tree $\longrightarrow$ non-linear DS

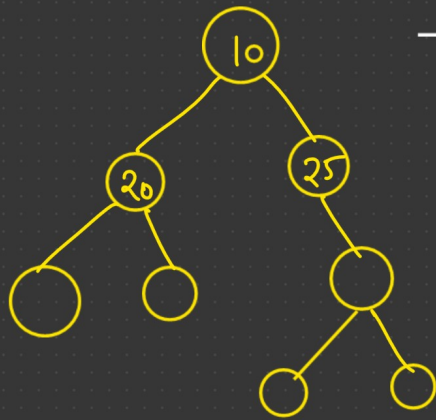




Binary Tree

Atmost 2 child

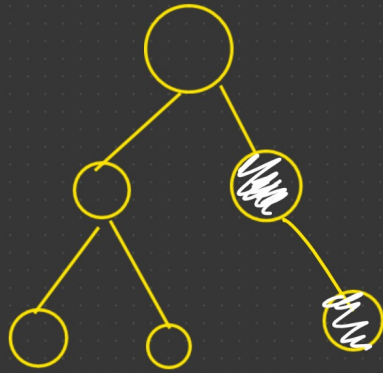
(0, 1, 2) $\therefore$ —————> No. of child node



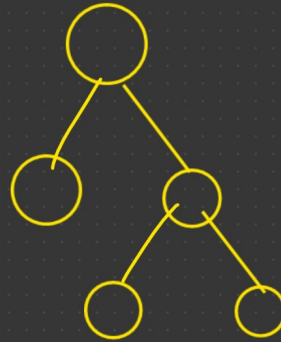
—————>	Level/Depth = 0	Height = 4
✓ —————>	Level/Depth = 1	Height = 3
—————>	Level = 2	Height = 2
—————>	Level = 3	Height = 1

Full Binary Tree / Strict Binary Tree:-

Exactly "0" or "2" child



X

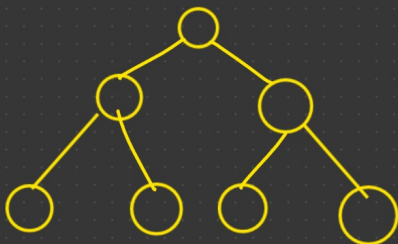


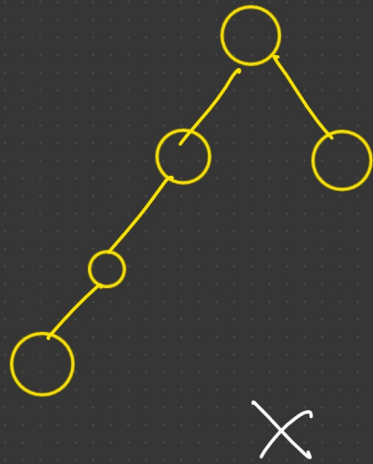
✓

Complete BT:- ① 0, 1, 2

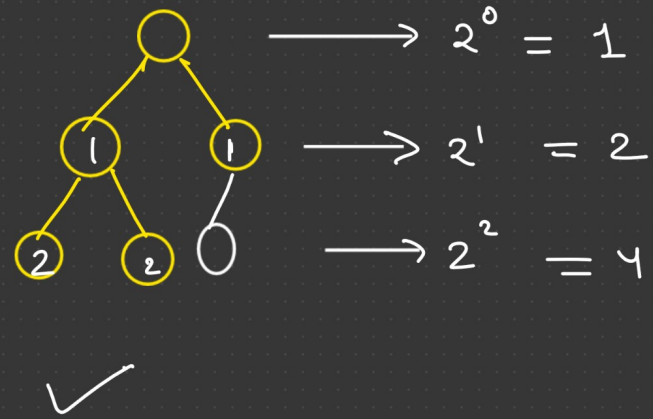
② But Left $\rightarrow$ right

③ first finish current level then move to next level.



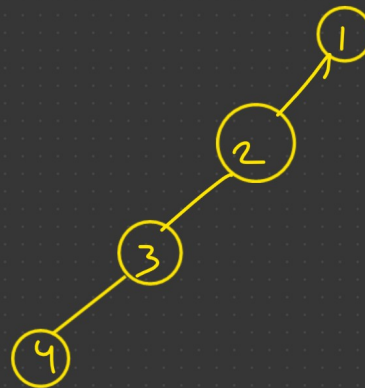
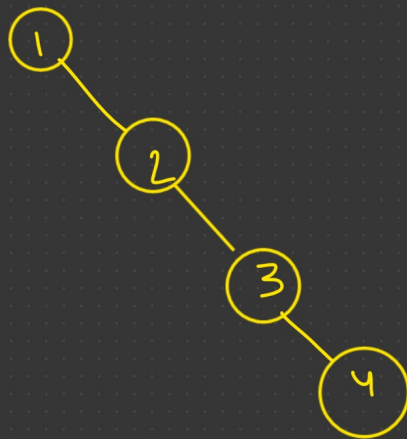


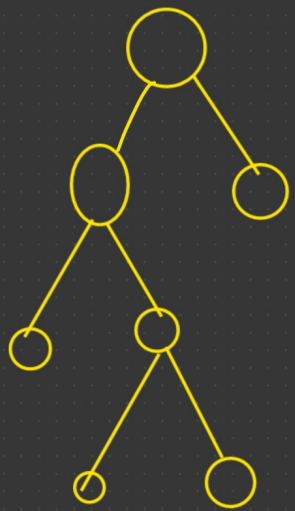
$$\begin{aligned} &\longrightarrow 2^0 \\ &\longrightarrow 2^1 \\ &\longrightarrow 2^2 \end{aligned}$$



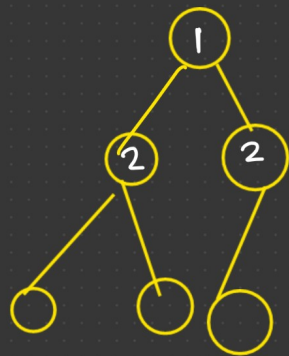
$$\begin{aligned} &\longrightarrow 2^0 = 1 \\ &\longrightarrow 2^1 = 2 \\ &\longrightarrow 2^2 = 4 \end{aligned}$$

Skew Tree :-

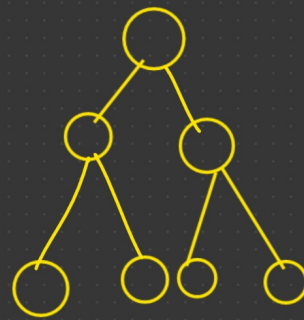




FBT ✓
CBT ✗



CBT ✓
FBT ✗



CBT ✓
FBT ✓

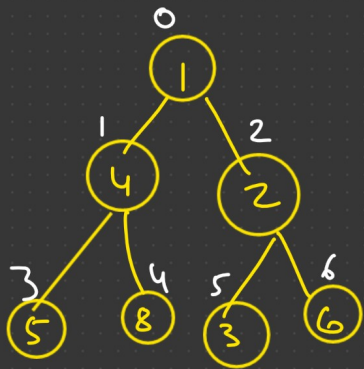


CBT ✗
FBT ✗

Binary Tree Representation:-

- 1) Array Representation
- 2) LL Rep.

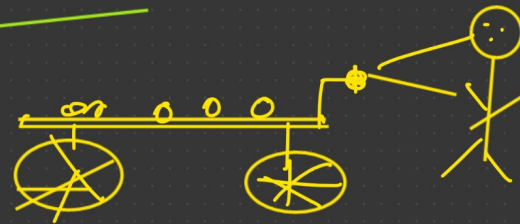
Array Representation

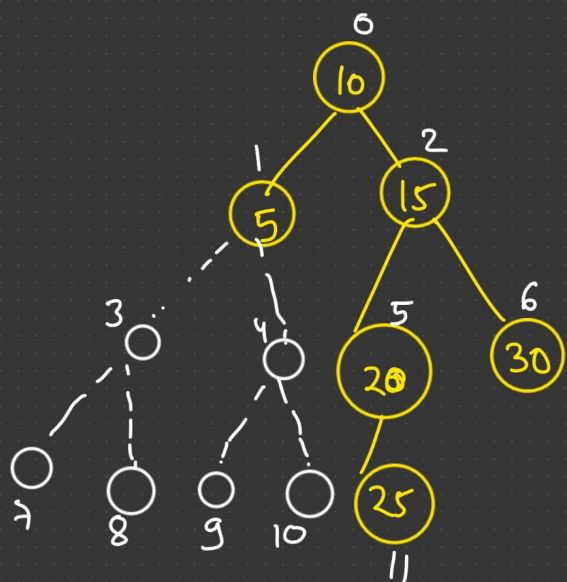


↑
Logical view

1	4	2	5	8	3	6		
0	1	2	3	4	5	6	7	8

↑
Physical view

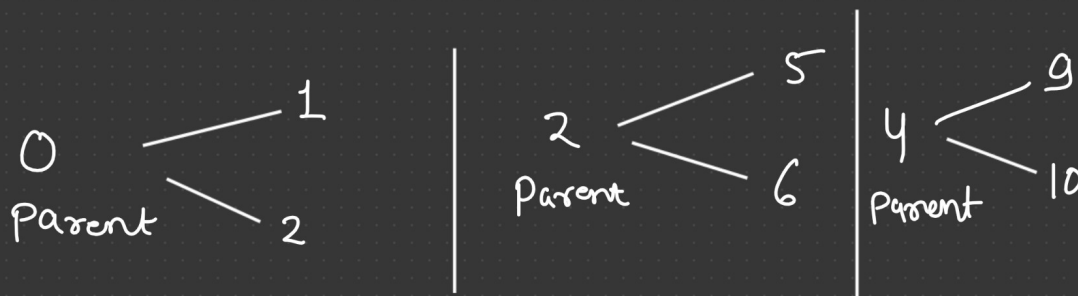




10	5	15			20	30					25		
0	1	2	3	4	5	6	7	8	9	10	11	12	13



↓
Size = 12



$$\begin{aligned} (\text{Parent} * 2) + 1 &= \text{left child} \\ (\text{Parent} * 2) + 2 &= \text{Right child} \end{aligned}$$

$$\left\lfloor \frac{\text{Left child}}{2} \right\rfloor = \text{parent}$$

$$\left\lfloor \frac{\text{Right child}}{2} - 1 \right\rfloor = \text{parent}$$

$$\lfloor 2 \cdot 9 \rfloor = 2$$

$$\lfloor 3 \cdot 1 \rfloor = 3$$

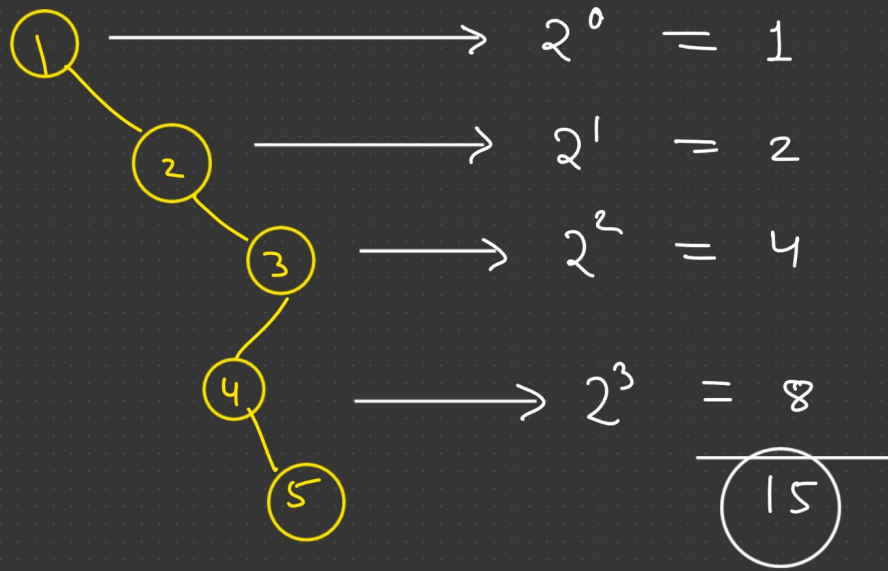
$$\lfloor 2 \cdot 9 \rfloor = 3$$

$$\lfloor 2 \cdot 17 \rfloor = 3$$

$$\lfloor 3 \cdot 1 \rfloor = 4$$

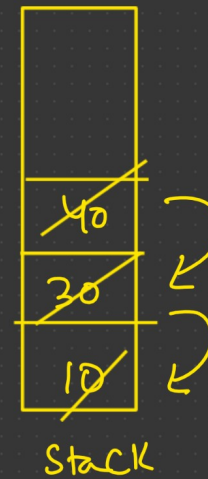
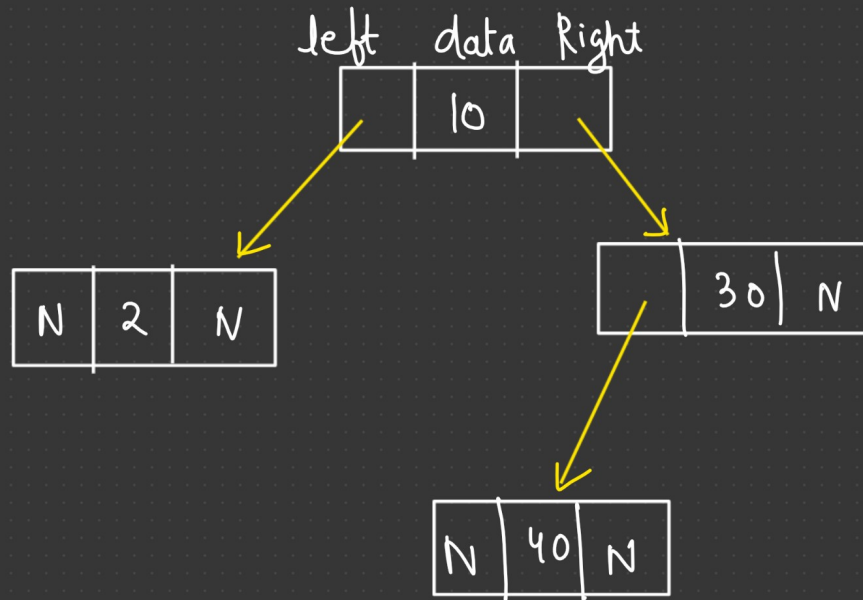
Drawback of Array Rep:-

Wastage of memory

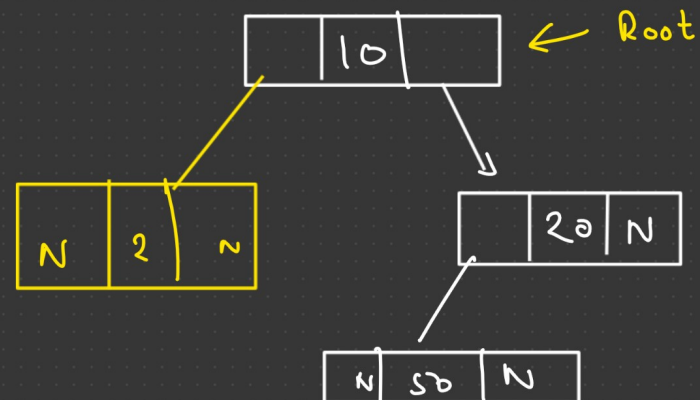


$$\begin{array}{r} \text{Size} = 14 \\ + 16 \\ \hline 30 \end{array}$$

Linked list Rep :-



Recursion




```
class node
```

```
{
```

```
    public :
```

```
        int value;
```

```
        node * left;
```

```
        node * right;
```

```
        node (int v)
```

```
{
```

```
    value = v;
```

```
    left = NULL;
```

```
    right = NULL;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    node * root = new node(10);
```

```
    root → right = new node(20);
```

```
    root → left = new node(2);
```

```
    root → right → left = new node(50);
```

```
    inorder(root);
```

```
    preorder(root);
```

```
    postorder(root);
```

```
    levelorder(root);
```

```
    return 0;
```

```
}
```

Testing
code

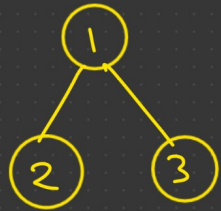
Tree Traversal

Array, stack, rec, LL.

```
while (node != NULL)
{
    node = node -> next;
}
```

```
{
    for (i=0; i<n; i++)
    {
}
```

```
while (stack() != empty)
{
    push()
}
```



① ①, 2, 3

→ preorder

(①, 2, 3)

② 2, ①, 3

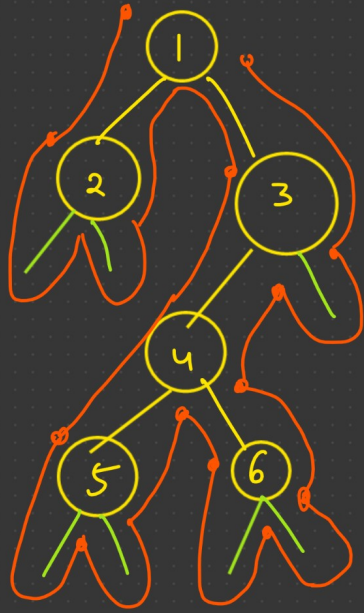
→ Inorder

(2, ①, 3)

③ 3, 2, ①

→ Postorder

(2, 3, ①)



- 1) 1, 2, 3, 4, 5, 6
- 2) 2, 1, 5, 4, 6, 3
- 3) 2, 5, 6, 4, 3, 1

1) Preorder :- Root $\rightarrow$ left subtree $\rightarrow$ Right subtree

1, 2, 3, 4, 5, 6.

2) Inorder :- left subtree $\rightarrow$ Root $\rightarrow$ Right subtree

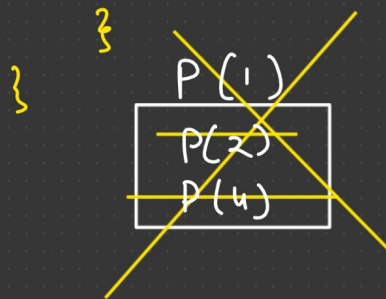
2, 1, 5, 4, 6, 3.

3) Post order :- left subtree $\rightarrow$ Right subtree $\rightarrow$ Root

2, 5, 6, 4, 3, 1

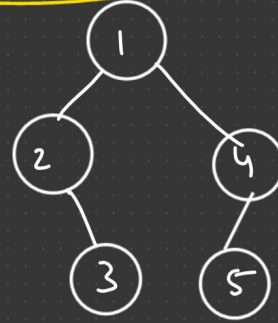
```
void preorder(root)
```

```
{  
  if (root != NULL)  
  {  
    cout << root->value;  
    preorder(root->left);  
    preorder(root->right);  
  }  
}
```



Root → left repeat → right repeat

1, 2, 3, 4, 5, ✓



```

void inorder(root)
{
    if (root != NULL)
    {
        preorder(root → left);
        cout << root → value;
        preorder(root → right);
    }
}

```

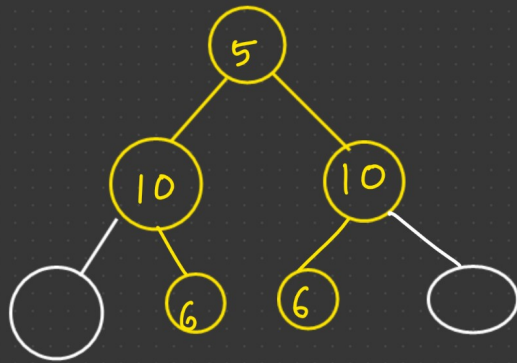
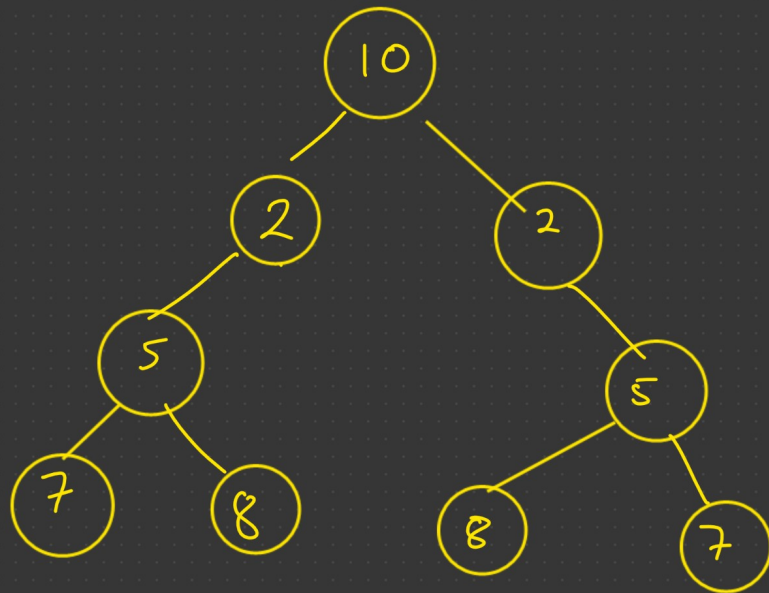
```

void postorder(root)
{
    if (root != NULL)
    {
        preorder(root → left);
        preorder(root → right);
        cout << root → value;
    }
}

```

1) Tree code

2) list code



Recursion :-

```
bool Symmetric (node *root)
```

```
{
```

```
if (root == Null) return true;
```

```
if (root->left == null && root->right == null)
    return true;
```

```
if (root->left == null && root->right != Null)
    return false; +1
```

```
return ((root->left->value == root->right->value) = T
&& (Symmetric (root->left))
&& Symmetric (root->right)); +1
```

3


```
bool symmetric (Node *root1, Node *root2)
```

```
{
```

```
    if (root1 == Null && root2 == Null)
```

```
        return true;
```

```
    if (root1 == Null || root2 == Null)
```

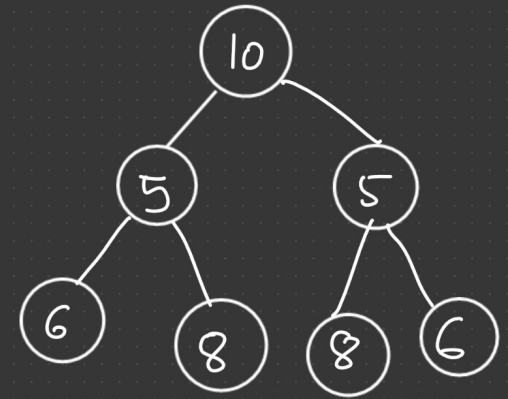
```
        return false;
```

```
    return ((root1->value == root2->value)
```

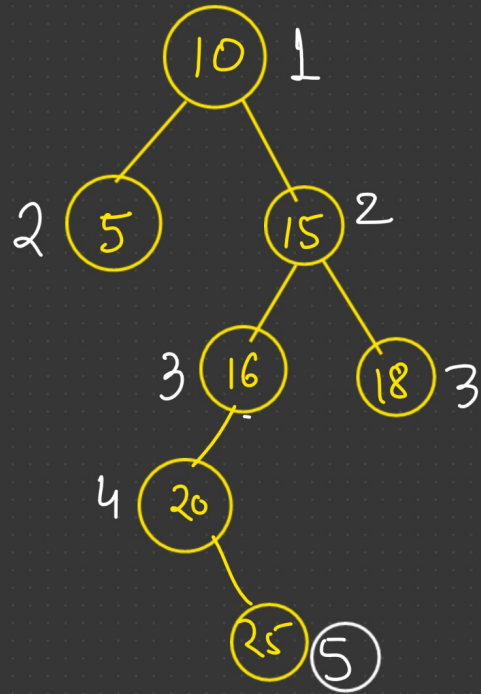
```
        && symmetric (root1->left, root2->right)
```

```
        && symmetric (root1->right, root2->left));
```

```
}
```



max - depth :-



$$1 + \max(\text{depth}(\text{root} \rightarrow \text{left}), \text{depth}(\text{root} \rightarrow \text{right}))$$

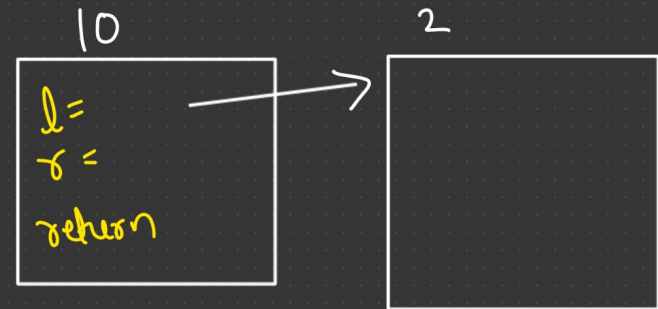
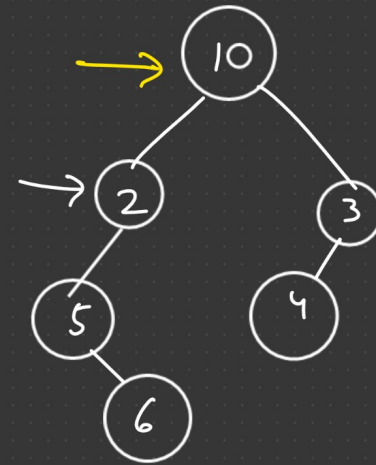
```
int depth (node *root)
```

```
{  
    if (root == null) {  
        return 0;  
    }
```

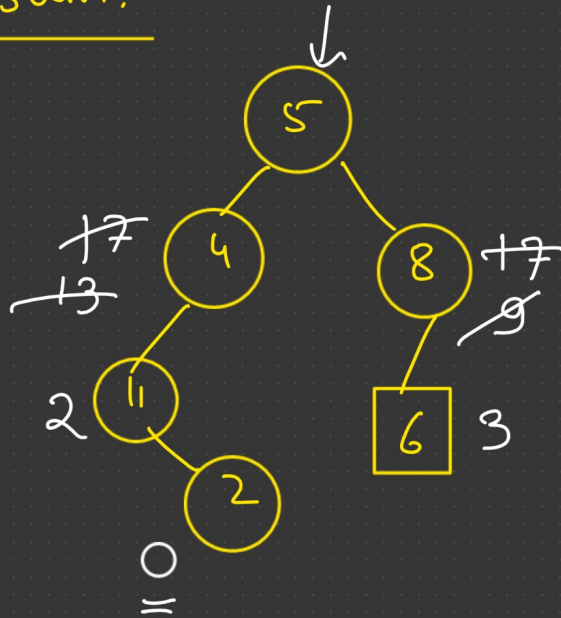


```
    int left = depth (root->left);  
    int right = depth (root->right);  
    return [1 + max(left, right)];  
}
```

Calculation



Path sum:-



True

```
bool pathsum(node*root, int TS)
{
    if (root == null && TS != 0)
        return false;

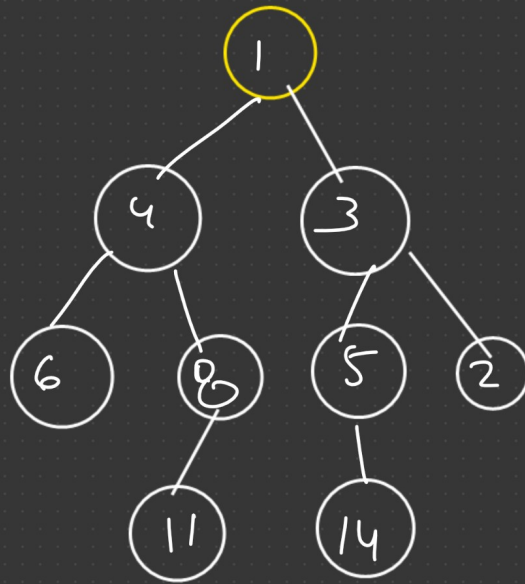
    if (root->left == null && root->right == null)
        return (TS == root->value);

    TS = TS - root->value;

    return (pathsum(root->left, TS)
        || pathsum(root->right, TS));
}
```

Level order Traversal:-

1, 4, 3, 6, 8, 5, 2, 11, 14



1	Null	4	3	Null	6	8	5	2	Null	11	14	Null
---	------	---	---	------	---	---	---	---	------	----	----	------

↓
①

↓
②

↓
③

↓
④

1, 4, 3, 6, 8, 5, 2, 11, 14

```
void levelorder (node * root)
```

```
{
```

```
    if (root == null)
```

```
        return;
```

```
    queue < node * > q;
```

```
    q.push (root);
```

```
    while ( ! q.empty() )
```

```
    {
```

```
        node * temp = q.front();    q.pop();
```

```
        cout << temp->value;
```

```
        if ( temp->left != NULL ) q.push ( temp->left );
```

```
        if ( temp->right != NULL ) q.push ( temp->right );
```

```
    }
```

```
}
```

BFS



temp = 1, 2, 3, 4, 5